Kenan FIRAT
Student Number: 2026820012
Git Repo : https://github.com/euphratesk/ds582_1.git

Question 1:

Zero Shot:

With zero shot prompting model have no idea to generate to response. It has a ability to grasp solution but it has no idea how it attends it without example and explicit operational structure (Zero Shot), the model fails to trigger its problem-solving (mathematical reasoning) capabilities. In small instruction-tuned models (like Qwen2.5-0.5B-Instruct), in this case because input formating does not exist which normally push model problem solving, greedy decoding (do_sample=False) on a minimal zero-shot prompt can trigger standard "safety/assistant boilerplates" . Without a ground in-context demostration, 0.5 Billion parameter model lacks sufficient guidance and It treats the prompt as an ambiguous query rather than a direct instruction to calculate. This structural deficit resulted in 0% accuracy (0/30).

Four Shot::

Feeding the model with diverse demonstration examples in an instructive step by step, model adjust its frozen parametric memory into mimic structured reasoning templates. Under frozen weight condition, in context examples tells to the model to bypass conversational filler and focus on intermediate numerical decomposition.This dramaticall improve model accuracy to 36%7 (11/30) while maintaining a highly efficient token footprint (8698 total tokens)

Sel-Consistency:

When we operate self connsistency method we make do_sample (do_sample =True) parameter activated.This allow the model explore different reasoning routes in contrast to Zero shot which model picks the single most likely token every step which created one deterministic path.On the other hand self consistency follows stocastic generating in a controlled randomness, due to we demand it 4 times. Small models as our model 0.5 billion parameter make minor arithmetic or logical slips on a single pass. With random sampling reasoning routes model manage to break out rigid conversational loops it is stuck which happen in zero shot. On the otherhand randomness cause more exploration and naturally generate more tokens cost engine resources. While stochasticity successfully prevents the model from stuck in rigid, incorrect deterministic loop, it cost severe computational cost (36385 tokens) and interestingly accuracy drop to 23.3% compare to Four Shot.

Reflection and Revised

Even though in this method model follows greedy decoding which is same parameter do_sample = False as Zero shot does, the context changes between call 1 and call 2 frying the model development run. At the first, shot model receives the raw prompt and generates tokens from scratch. At the second, shot model contains original question, a generated solution draft and with the prompt instruction reviewing the solution step by step against to zero shot method. Through transformer self attention mechanism with its gains from the prior call the, attending over its own generated draft alters the output token probablitiy distribution. Model has experience and no longer to try to solve the problem from scratch. The model adjusts its prior generation within the context window to evaluate individual steps. This narrows the solution

space,ensure model acts its own evaluator,using context driven attention to catch potential arithmetic slip and harness the self attention mechanics to focus on useful tokens. While this method refine several instance, total token consumption doubled (19047) against to Four Shot).Model achieved 30% accuracy (9/30)

Final Conclusion:
Four_shot provides the optimal accuracy-cost tradeoff. It achieves the highest test accuracy (%36.7) while consuming less than half the tokens of Reflection and a Revised and a fraction of Self Consistency (8698,19047,36385). IN lightweight architectures like Qwen2.5-0.5-Instruct, static structural guidance via few shot examplars vastly outperforms expensive compute scaling methods (stochastic sampling or multi-turn reflection)

Comparison for Prompting Method

	Model Calls Per Question	Total Correct	Total False	Total Tokens	Avg. Tokens Per Question	Avg. Tokens For Correct Answer	Avg. Tokens For False Answer
Few Shot	1	11	19	8698	289.93	259.73	307.42
Reflection	2	9	21	19047	634	590.78	653.81
Self-Consistency	4	7	23	36385	1212.83	1101.71	1246.65
Zero Shot	1	0	30	2400	80	0	80

Question 2:

In accordance wıth the task description “Build an agent that translates each question into arithmetic expressions, calls a restricted Python calculator, observes its results or errors, and produces a final answer”, a multi-turn ReACt framework was developed. The agent utilizes a lightweight text-trigger protocol (Calculate) combined with a secure AST-based interpreter (enforcing strict ceiling of three calls per question and blocking unrestricted execution) to parse and evaluate model-generated arithmetic expressions dynamically/

Comparison Best Prompting Method and Agent

	Model Calls Per Question	Total Correct	Total False	Total Tokens	Avg. Tokens Per Question	Avg. Tokens For Correct Answer	Avg. Tokens For False Answer
Few Shot	1	11	19	8698	289.93	259.73	307.42
React Agent	1	1	29	3191	106.27	229	102.14